

NATIONAL ECONOMICS UNIVERSITY

FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE



BUILDING A LIVE AUCTION BIDDING SYSTEM: DESIGN AND OPERATION

Course: Network and Communication Technologies

Team Members: 11250640 - Nguyen Ngoc Anh

11255166 - Pham Ngoc Mai

11256961 - Nguyen Ha Phuong

Class: Data Science 67A

Date: September 21, 2026

Abstract

This report presents the design and implementation of a Live Online Auction Bidding prototype based on the Client-Server architecture. The primary objective is to simulate a real-time auction environment where multiple participants can connect, receive live updates, and submit bids simultaneously. The system is developed using Python TCP Sockets for reliable network communication and multi-threading techniques to handle concurrent client connections. To prevent race conditions and ensure data consistency, thread synchronization mechanisms (Locks) are applied to manage the shared “highest bid” variable. Additionally, the prototype integrates an automated 15-second count-down timer that resets upon receiving a valid higher bid and automatically declares the winner when the time expires. The experimental results, validated through various testing scenarios, demonstrate that the system successfully manages concurrent user interactions, accurately synchronizes shared states, and provides a stable real-time bidding experience.

The complete source code is publicly available on Github. For empirical evidence of the system’s performance, including stress test logs and UI behavior during race conditions, please refer to our full Test Result & Evidence

Acknowledgement

First and foremost, we would like to express our deepest gratitude to Dr. Tran Duc Minh, our lecturer, for his insightful teaching and for providing us with the opportunity to undertake this project. His guidance has been fundamental to our academic growth throughout this course.

We would also like to extend our sincere thanks to National Economics University for providing a dynamic learning environment and the necessary resources that greatly facilitated our research and academic journey.

Furthermore, we want to express our appreciation to all team members for their hard work, effective collaboration, and dedication throughout this project. Working together to handle complex multi-threaded synchronization and TCP socket communication has been an incredibly rewarding experience.

Despite our best efforts, we acknowledge that this report may still have certain limitations due to our practical inexperience. We warmly welcome any feedback and constructive criticism from you to help us improve in the future.

Contents

List of Figures	4
List of Listings	4
1 Introduction	5
2 Theoretical Background	6
2.1 TCP/IP Protocol and Client-Server Model	6
2.2 Multi-threading	6
2.3 Race Conditions and Resource Synchronization	7
3 System Architecture & Design	9
3.1 System Block Diagram	9
3.2 Application-layer Protocol	9
3.3 Shared Data Structures and Resource Synchronization	11
3.4 Multi-threaded Operational Flow & Scalability Analysis	11
3.5 Message Sequence of an Auction Session	13
4 Implementation Details	15
4.1 Socket Initialization & Signal Handling	15
4.2 Safe Dictionary Iteration during Broadcasts	15
4.3 Critical Section Synchronization (Check-then-Act Pattern)	16
4.4 Timer Cancellation Race Mitigation	16
4.5 Client Implementation	17
5 Testing & Results	20
5.1 Test Environment	20
5.2 Test case 1: Connection & Bid Validation	20
5.3 Test Case 2: Timer Reset Mechanism	21
5.4 Test Case 3: Concurrency & Stress Testing	22
5.5 Test Case 4: Fault Tolerance (Client Disconnection)	23
5.6 Test Case 5: Message Framing over TCP	25
6 Conclusion	27
6.1 Achievements	27
6.2 Technical Challenges & Solutions	27
6.3 Known Limitations	28
6.4 Future Work	29
6.5 Concluding Remarks	29

List of Figures

1	Centralized Star Topology of the Auction System	9
2	Message sequence of an auction session with two bidders	14
3	Bid validation responses (ERROR and UPDATE)	21
4	The countdown timer explicitly resetting from 4s back to 15s.	22
5	Stress test results showing successful synchronization of 3,000 concurrent bids.	23
6	Server executing graceful degradation after a client disconnects unexpectedly (case a).	24
7	Remaining clients successfully resolving the final winner	24
8	Server logs showing process termination handled gracefully without error, and countdown continuing (case b).	25
9	Message framing test: sticky packets, fragmented packets and a split UTF-8 character (right), and the corresponding server log (left).	26

List of Listings

1	Socket Initialization	15
2	Safe dictionary iteration with lock	15
3	Check-then-Act synchronization for bidding	16
4	Mitigating timer cancellation race condition via version control	16
5	Client receiver loop: framing, dispatch and prompt handling (abridged)	17

Chap 1

Introduction

This report presents the design and implementation of a Live Online Auction Bidding System, developed as our midterm project for the Network & Communication Technology course. Simulating a real-time auction provides a highly practical scenario to apply core computer networking concepts, specifically focusing on reliable data transmission and concurrent processing. To achieve this, our team focused on resolving the following key technical requirements as specified in the assignment:

- **TCP Socket Programming:** Establishing a reliable communication protocol between the central Server (Auctioneer) and multiple Clients (Bidders).
- **Multi-threaded Architecture:** Implementing multi-threading to handle multiple user connections simultaneously without blocking the system.
- **Shared Variable Synchronization:** Applying thread synchronization (Locks) to safely manage the shared "highest bid" variable, preventing race conditions during concurrent bid submissions.
- **Real-time Countdown Logic:** Developing a dynamic 15-second countdown timer that resets upon receiving a valid higher bid and concludes the auction automatically.

Chap 2

Theoretical Background

2.1 TCP/IP Protocol and Client-Server Model

The system follows the Client-Server model, in which a central server (the Auctioneer) maintains the authoritative auction state and multiple clients (the Bidders) connect to it to submit bids and receive updates.

TCP was chosen over UDP as the transport protocol because an auction cannot tolerate lost, duplicated, or reordered messages: a dropped bid or a missing price update would make the auction unfair. TCP provides connection-oriented, reliable, and in-order delivery through its three-way handshake, acknowledgements, and retransmission mechanisms. On the server side, a connection is established through the standard socket lifecycle (`socket()` → `bind()` → `listen()` → `accept()`), while each client simply calls `connect()`.

However, TCP operates as a continuous byte stream without message boundaries: several application messages may arrive in a single `recv()` call, or one message may be split across several calls. Consequently, the application layer must implement its own message framing mechanism (described in Section 3.2) to correctly separate and process each message.

2.2 Multi-threading

A thread is an independent flow of execution inside a process. Unlike separate processes, all threads of a program share the same memory space, which makes exchanging data between them cheap but also means that any variable can be read or modified by several threads at once.

Network servers need concurrency because the core socket operations are *blocking*: `accept()` waits until a new client connects, and `recv()` waits until data arrives. In a single-threaded server, waiting on one client would stall every other client. Common approaches to this problem include:

- **Thread-per-connection:** each client is served by its own thread, which may block freely without affecting others. This is simple to write and reason about, but every thread consumes memory and adds context-switching overhead.
- **I/O multiplexing / event loop:** a single thread monitors many sockets using

`select`, `epoll` or `asyncio`, and only handles those that are ready. This scales to far more connections, at the cost of more complex, non-blocking code.

Our system adopts the thread-per-connection model, which is well suited to the scale of an auction room and keeps the bidding logic sequential and readable (its scalability limits are discussed in Section 3.4). Threads are used in three roles:

- **Server connection threads:** the main thread only runs the `accept()` loop, and spawns one `handle_client` thread per bidder.
- **Server countdown threads:** the 15-second timer runs on background threads, so that time keeps passing independently of any client's activity.
- **Client threads:** `client.py` separates user input from network reception. The main thread blocks on keyboard input, while a background thread continuously receives `UPDATE` and `WIN` broadcasts, so a bidder sees new prices in real time even while typing.

All background threads are created as *daemon* threads, so they are terminated automatically when the main program exits and cannot keep the process alive after shutdown.

2.3 Race Conditions and Resource Synchronization

A **race condition** occurs when the correctness of a program depends on the relative timing of threads that access shared data. The code region that accesses such data is called a **critical section**. In this system, the critical sections involve the highest bid and leading bidder, the dictionary of connected clients, and the countdown state.

The most dangerous pattern is **check-then-act**: a thread first checks a condition and then acts on it, assuming the condition still holds. Bid validation follows exactly this pattern (*if the new amount is higher, then update the highest bid*). Table 1 shows how, without synchronization, two concurrent bids can produce a *lost update*.

Table 1: A lost update when two bids are validated without a lock (current highest bid: 1,000,000)

Step	Thread A (bid 3,000,000)	Thread B (bid 2,000,000)
1	Reads highest bid: 1,000,000	
2		Reads highest bid: 1,000,000
3	3,000,000 > 1,000,000: writes 3,000,000	
4		2,000,000 > 1,000,000: writes 2,000,000
Result: highest bid is 2,000,000, although a higher bid of 3,000,000 was accepted.		

A **mutual exclusion lock (mutex)** prevents this interleaving: only one thread can

hold the lock at a time, so the check and the update execute as one indivisible step. In the example above, thread B would have to wait until thread A released the lock, and would then correctly find that 2,000,000 is lower than the new highest bid.

Why locks are needed despite the GIL. CPython’s Global Interpreter Lock (GIL) allows only one thread to execute Python bytecode at a time, which is sometimes mistaken for built-in thread safety. It is not: the interpreter can switch threads between any two bytecode instructions, so a compound operation such as check-then-act, or even `x += 1`, can still be interleaved. The GIL is also released during blocking I/O such as `recv()`, which is precisely what allows threads to handle clients concurrently. Moreover, recent Python versions offer an optional free-threaded build without a GIL at all. Explicit locks are therefore required for correctness regardless of the GIL.

Design principles applied in this system.

- **Lock separation:** the auction state is guarded by `price_lock` and the client list by `room_lock`, so a bid does not block a client from joining or leaving, and vice versa (Section 3.3).
- **Short critical sections:** slow network I/O, such as broadcasting an `UPDATE`, is performed after `price_lock` is released, so other bidders are not kept waiting (Section 4.3).
- **Deadlock avoidance:** a deadlock can occur when two threads each hold one lock while waiting for the other’s. The system rules this out by never holding both locks at the same time: every code path releases one lock before acquiring the other.
- **Re-validation under the lock:** any decision based on a value read outside a lock must be checked again once the lock is held. This principle is the basis of the timer race fix described in Section 4.4.

Chap 3

System Architecture & Design

3.1 System Block Diagram

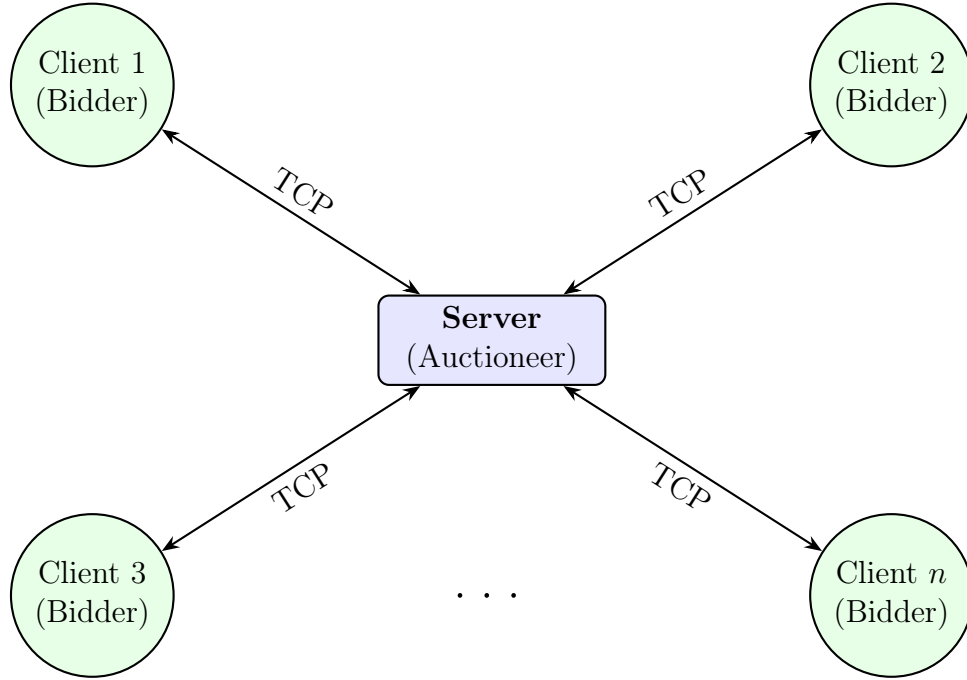


Figure 1: Centralized Star Topology of the Auction System

Client-Server architecture utilizing the TCP (Transmission Control Protocol) to ensure reliable, ordered, and error-checked delivery of data. The Server acts as the central Auctioneer, concurrently managing connections with n Clients (Bidders) in a star topology.

3.2 Application-layer Protocol

The client and server communicate using a custom text-based protocol encoded in UTF-8. Each message consists of a command followed by fields separated by the pipe character (`|`), and is terminated by a newline character (`\n`), which serves as the message delimiter for framing (see *Message Framing* below). Table 2 lists all message types.

Table 2: Message types of the auction protocol

Message	Direction	Purpose
JOIN <username>	Client → Server	Joins the auction room with a unique, non-empty username.
BID <amount>	Client → Server	Submits a bid; amount must be a positive integer.
WELCOME <item> <price>	Server → Joiner	Confirms the JOIN and announces the item and its starting price.
UPDATE <price> <leader>	Server → Client(s)	Reports the current highest bid and leading bidder. Sent to a new client right after WELCOME, and broadcast to every client whenever a bid is accepted.
INFO <message>	Server → Others	Notifies the other clients that a new bidder has joined.
WIN <winner> <price>	Server → All	Announces the result when the countdown expires (WIN none 0 if no bid was placed).
ERROR <reason>	Server → Sender	Rejects a request; the reason is listed in Table 3.

A client leaves the room simply by closing its TCP connection; the server detects the closed connection and removes the client (Section 5.5). Table 3 lists every condition under which the server replies with an ERROR message.

Table 3: Conditions that trigger an ERROR reply

Trigger	Meaning
JOIN (repeated)	The client has already joined.
JOIN (empty name)	The username is empty after trimming.
JOIN (duplicate name)	Another client already uses this username.
BID before JOIN	The client must join before bidding.
BID (not an integer)	The amount cannot be parsed as an integer.
BID (amount ≤ 0)	The amount must be positive.
BID (too low)	The amount is not strictly higher than the current highest bid; the reason includes the current price.
JOIN/BID after end	The auction has already concluded.
Unknown command	The message does not match any command or lacks a required field.

The following exchange illustrates a valid bid, as seen on the wire:

Client → Server : BID|3000000

Server → All : UPDATE|3000000|na

- **Message Framing & Defense Mechanisms:** Because TCP is a stream-oriented protocol, consecutive `send()` calls can coalesce into single segments under high network loads (sticky packets). The `\n` delimiter enables the recipient's buffer to accurately isolate and parse discrete instructions.

- **Input Sanitization & Buffer Defense:** All incoming inputs are strictly validated.

3.3 Shared Data Structures and Resource Synchronization

To manage shared states safely without causing performance bottlenecks, the system employs a disciplined lock-separation strategy:

- `highest_bid`, `highest_bidder`, and the `auction_active` flag are strictly protected by a mutex named `price_lock`.
- The `clients` dictionary mapping active sockets to usernames is protected by a dedicated `room_lock`, preventing race conditions during insertions, removals, and broadcast iterations.
- The `timer_version` counter is protected by `price_lock`: it is incremented inside `reset_timer()`, which is always called within the lock, and re-validated under the same lock before the auction is closed.

3.4 Multi-threaded Operational Flow & Scalability Analysis

Building on the concurrency model introduced in Section 2.2, this section describes how the threads of the system cooperate at runtime, and then analyses how the design behaves as the number of bidders grows.

Server-side flow.

- **Main thread:** runs the `accept()` loop. The listening socket uses a one-second timeout, so the loop wakes up regularly to check the `auction_active` flag and stops accepting connections within one second after the auction closes. Each accepted connection is handed to a new `handle_client` thread, after which the main thread immediately returns to waiting for the next client.
- **Client handler threads** (one per bidder): repeatedly call `recv()`, reassemble complete messages from the byte stream (Section 3.2), and process `JOIN` and `BID` commands. When the connection closes, the thread removes its client from the room and terminates.
- **Countdown threads:** the first `JOIN` and every accepted bid call `reset_timer()`, which starts a new countdown thread tagged with the current `timer_version`. A superseded thread notices the version change on its next one-second tick and exits, so only the most recent countdown can close the auction (Section 4.4).

Client-side flow. After the user enters a name, `client.py` connects to the server with a five-second connection timeout, so an unreachable or mistyped server address

produces a clear error message instead of an indefinite hang. It then sends **JOIN** and runs two threads:

- **Main thread:** blocks on keyboard input, performs basic local validation (the amount must be a positive integer), and sends each valid amount as a **BID**. Typing **quit**, **exit** or **q** closes the connection. The server still re-validates every bid, so the local check only saves a round trip for obvious input mistakes.
- **Receiver thread:** a daemon **receive_messages** thread continuously reads from the socket, reassembles messages using the same newline framing and incremental UTF-8 decoding as the server, and displays **WELCOME**, **UPDATE**, **INFO**, **ERROR** and **WIN** messages as soon as they arrive. After each batch of messages it re-prints the input prompt once, so the prompt is not duplicated when several messages arrive together.

Without this separation, a bidder would not see a competitor’s new price until they finished typing their own bid. The two threads coordinate through a shared **running** flag; because it is a single boolean that is only switched from **True** to **False**, no lock is needed. One limitation remains: a blocking **input()** call cannot be interrupted by another thread, so when **WIN** arrives the client asks the user to press Enter once more before the program exits.

Table 4 summarizes all threads, how many of each exist, and which shared state they access.

Table 4: Threads in the running system

Thread	Count	Lifetime	Locks used
Server main	1	Whole server run	room_lock (at shutdown)
handle_client	1 per bidder	While connected	price_lock (JOIN, BID), room_lock (join, broadcast, cleanup)
Countdown	1 per reset	Up to 15 s; exits within 1 s once superseded	price_lock (only when closing the auction)
Client main	1 per client	Whole session	none (only writer to the socket)
Client receiver	1 per client	Whole session	none (only reader of the socket; shares the boolean running flag)

Scalability analysis. The thread-per-connection model is adequate for an auction room of tens of bidders, as confirmed by the stress test in Section 5.4: 30 concurrent clients submitted 3,000 bids, which the server processed in 3.27 seconds (roughly 900 bids per second) with the correct final price. As the number of bidders n grows, however, four costs increase:

- **Thread overhead:** the server holds at least n handler threads, each with its own stack, and the operating system must switch between them. With thousands of clients (the C10K problem), memory use and context-switching overhead dominate.
- **Broadcast fan-out:** every accepted bid is sent to all n clients, so k accepted bids generate $k \times n$ outgoing messages. Because `send_to_all()` sends to clients one after another while holding `room_lock`, a single slow client can delay the broadcast to everyone else and block joins and disconnections in the meantime.
- **Countdown thread churn:** since each accepted bid spawns a new countdown thread and a superseded one may live for up to one more second, a burst of r accepted bids per second keeps roughly r countdown threads alive at once.
- **Serialization on `price_lock`:** all bids are validated one at a time. This is inherent to an auction, since bids must be totally ordered to decide the highest one, so it is a correctness requirement rather than a design flaw; keeping the critical section short (Section 4.3) is what limits its cost.

For substantially larger deployments, these costs point to three improvements: replacing per-client threads with I/O multiplexing (`selectors` or `asyncio`), giving each client its own outgoing message queue so that a slow receiver cannot stall a broadcast, and replacing per-bid countdown threads with a single timer thread that compares the current time against a shared deadline.

3.5 Message Sequence of an Auction Session

Figure 2 shows the messages exchanged during a typical session with two bidders. The countdown starts when the first client joins and is reset to 15 seconds on every accepted bid; the auction ends when 15 seconds pass without a higher bid.

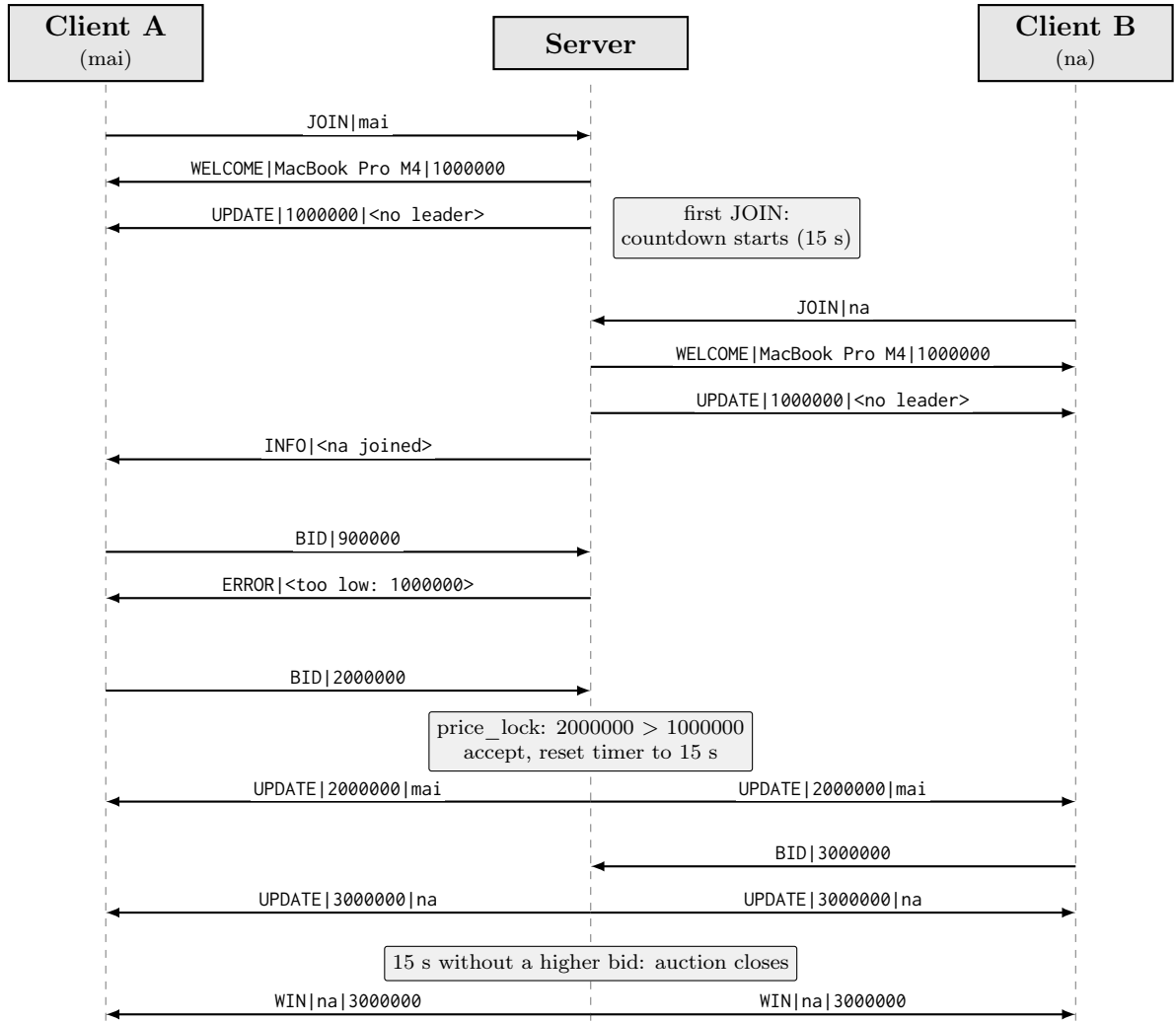


Figure 2: Message sequence of an auction session with two bidders

Chap 4

Implementation Details

4.1 Socket Initialization & Signal Handling

To eliminate binding failures (such as the *Address already in use* error caused by sockets stuck in `TIME_WAIT`), the `SO_REUSEADDR` flag is explicitly configured. Additionally, custom signal handlers for `SIGINT` and `SIGTERM` ensure uniform and graceful resource cleanup.

Listing 1: Socket Initialization

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((HOST, PORT))
server_socket.listen(10)
```

4.2 Safe Dictionary Iteration during Broadcasts

When broadcasting messages via `send_to_all`, the server iterates over the shared `clients` dictionary. This loop is safely enclosed within `room_lock` to prevent a fatal `RuntimeError: dictionary changed size during iteration` if a client disconnects mid-broadcast.

Listing 2: Safe dictionary iteration with lock

```
def send_to_all(message, exclude=None):
    disconnected = []
    with room_lock:
        for sock, username in clients.items():
            if sock == exclude:
                continue
            if not send_message(sock, message):
                disconnected.append(sock)
        for sock in disconnected:
            clients.pop(sock, "unknown")
            sock.close()
```

4.3 Critical Section Synchronization (Check-then-Act Pattern)

The core concurrency defense lies in bid validation. Evaluating `amount > highest_bid` and updating `highest_bid` forms a classic Check-then-Act pattern. This compound operation must execute atomically within `price_lock`.

Listing 3: Check-then-Act synchronization for bidding

```
with price_lock:
    if not auction_active:
        reason = "Auction_has_already_concluded"
    elif amount > highest_bid:
        highest_bid = amount
        highest_bidder = username
        accepted = True
        reset_timer() # Timer re-allocation safely handled inside lock
    else:
        reason = f"Bid_too_low_(current:_{highest_bid})"
```

Performance Optimization: Network I/O operations such as `send_to_all()` are intentionally executed outside `price_lock` to keep the critical section short and avoid blocking other bidding threads.

4.4 Timer Cancellation Race Mitigation

Each accepted bid calls `reset_timer()` inside `price_lock`, which increments `timer_version` and starts a new countdown thread tagged with that version. Superseded threads detect the mismatch and exit, so only the latest thread may close the auction.

However, in the initial implementation, the version was checked outside the lock, before `end_auction()` acquired it. If a bid was accepted in that short window, the stale thread would still close the auction immediately after the UPDATE was broadcast, denying other bidders their 15-second window. Checking `auction_active` alone cannot prevent this, because the bidding path never changes that flag.

The fix is to pass the thread's version into `end_auction()` and re-check it inside `price_lock`:

Listing 4: Mitigating timer cancellation race condition via version control

```
def end_auction(version):
    global auction_active
    with price_lock:
        if not auction_active or timer_version != version:
            return # already closed, or superseded by a newer bid
        auction_active = False
```

```
winner, final_price = highest_bidder, highest_bid
send_to_all(f"WIN|{winner}|{final_price}")
```

Since the version is both incremented and validated under the same lock, only two outcomes are possible: either the bid is accepted and the countdown restarts, or the auction closes first and the late bid is rejected. A bid can no longer be accepted and then immediately followed by the auction closing.

4.5 Client Implementation

The client (`client.py`) is intentionally thin: all auction logic and state live on the server, and the client only collects user input, transmits it, and displays what the server reports. Its implementation nevertheless has to solve three problems: connecting robustly, receiving messages while the user is typing, and shutting down cleanly in every situation.

Connection setup. After the user enters a name, the client creates a TCP socket with a five-second timeout for `connect()`. Timeouts, refused connections and other socket errors are reported with a hint (“Is the server running? Is the IP/port correct?”) instead of leaving the program hanging on a wrong address. Once connected, the socket is switched back to blocking mode, a `JOIN` message is sent, and the receiver thread is started.

Separating input from reception. Python’s `input()` blocks the calling thread until the user presses Enter. If the same thread also read from the socket, a competitor’s new price would only appear after the user had finished typing, defeating the purpose of a live auction. The client therefore uses two threads:

- **Main thread:** reads the bid amount, validates it locally (it must be a positive integer), and sends it as `BID|<amount>`. The server re-validates every bid, so this check only saves a round trip for obvious typing mistakes.
- **Receiver thread** (`receive_messages`): reads from the socket, reassembles complete messages, and displays them immediately (Listing 5).

The receiver applies the same framing as the server: incoming bytes pass through an incremental UTF-8 decoder, which holds back an incomplete multi-byte character (for example, part of a Vietnamese name in an `INFO` message) until the rest arrives, and the text is then split on newline characters. Because several messages can arrive in one `recv()` call, the input prompt is re-printed only once per batch rather than once per message.

Listing 5: Client receiver loop: framing, dispatch and prompt handling (abridged)

```

decoder = codecs.getincrementaldecoder('utf-8')()
while running:
    data = sock.recv(1024)
    if not data:                                # server closed the connection
        running = False
        break
    buffer += decoder.decode(data)               # keeps split UTF-8 bytes pending
    printed_something = False
    while '\n' in buffer:                       # one iteration per complete message
        line, buffer = buffer.split('\n', 1)
        parts = line.strip().split('|')
        command = parts[0]
        printed_something = True
        # ... display WELCOME / UPDATE / ERROR / INFO ...
        if command == "WIN" and len(parts) >= 3:
            # ... display the winner ...
            running = False                     # tells the main thread to stop
            break
    if printed_something and running:
        print(PROMPT, end='', flush=True)      # re-print the prompt once

```

Coordinating the two threads. The threads share a single boolean flag, `running`. The flag only ever changes from `True` to `False`, and a stale read merely delays shutdown by one loop iteration, so no lock is required. The socket itself is not contended either: the main thread is its only writer and the receiver thread its only reader.

Termination paths. Table 5 lists every way the client can stop. All of them converge on the same `finally` block, which clears the flag and closes the socket, so the server always observes an orderly connection close.

Table 5: Client termination paths

Trigger	Client behaviour
WIN received	Receiver displays the result and clears <code>running</code> ; the main thread exits after the user presses Enter.
Server closes connection	Receiver gets an empty <code>recv()</code> , reports it, and clears <code>running</code> .
Connection error	Receiver catches the exception, reports lost connection, and clears <code>running</code> .
quit / exit / q	Main thread leaves its loop.
Ctrl+C	<code>KeyboardInterrupt</code> is caught in the main thread.
<code>sendall()</code> fails	Main thread reports that the server may be down and leaves its loop.

Limitation. A blocking `input()` call cannot be interrupted by another thread. When the auction ends, the receiver can clear the flag but cannot wake the main thread, so the client explicitly asks the user to press Enter once more to exit. Likewise, the first input prompt may appear before the `WELCOME` message, since the two threads print to the console independently. A graphical interface driven by an event loop would remove both issues (Section 6.4).

Chap 5

Testing & Results

5.1 Test Environment

- **Operating System:** MacOS
- **Interpreter:** Python 3.13 (Conda Environment)
- **Testing scripts:** Custom Python automation script (`stress_test.py`) with `threading.Barrier` for concurrent bot generation.

5.2 Test case 1: Connection & Bid Validation

- **Operation:** To verify that the server accurately validates incoming bids, rejecting invalid ones (lower or equal to the current price) and accepting valid ones.
- **Execution:** Two clients (phuong and mai) joined the room (starting price: 1,000,000 VND). phuong submitted a lower bid (10,000 VND) and an equal bid (1,000,000 VND); mai submitted a lower bid (100,000 VND). mai then bid 1,900,000 VND, and phuong bid 3,000,000 VND.
- **Result:** All three invalid bids were rejected with `ERROR` messages. Both valid bids were accepted and broadcast as `UPDATE` messages to every client in the room. When the countdown expired, phuong was declared the winner at 3,000,000 VND.
- **Evidence:**

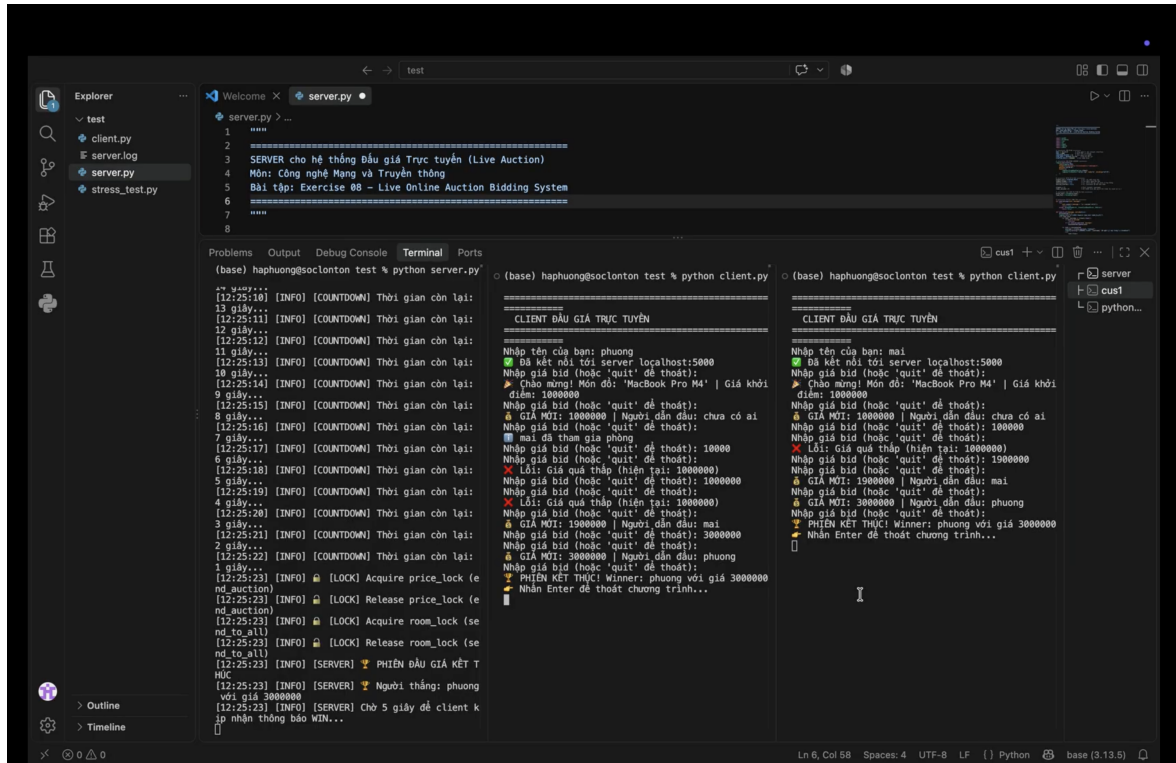


Figure 3: Clients receiving ERROR responses for invalid bids and UPDATE broadcasts for valid bids.

5.3 Test Case 2: Timer Reset Mechanism

- **Operation:** To ensure that the 15-second countdown timer accurately resets to its initial state upon receiving a valid bid.
- **Execution:** The countdown started when the first client joined. Client A bid 2,000,000 VND; when the countdown reached 4 seconds, Client B bid 3,000,000 VND.
- **Result:** The server instantaneously acquired the `price_lock`, registered the new bid, and reset the countdown timer back to 15 seconds. The console log clearly demonstrated the sequence jumping from 4 seconds back to 15 seconds.
- **Evidence:**

```
Problems Output Debug Console Terminal Ports
(base) haphuong@soclonlon test % python server.py
[12:25:03] [INFO] [COUNTDOWN] Thời gian còn lại: 4 giây...
[12:25:04] [INFO] [COUNTDOWN] Thời gian còn lại: 3 giây...
[12:25:04] [INFO] [LOCK] Acquire price_lock (B ID)
[12:25:04] [INFO] [TIMER] Timer đã được RESET về 15s
[12:25:04] [INFO] [COUNTDOWN] Thời gian còn lại: 15 giây...
[12:25:04] [INFO] [LOCK] Release price_lock (B ID)
[12:25:04] [INFO] [SERVER] [✓] Chấp nhận BID 1900000 từ 'mai'
[12:25:04] [INFO] [LOCK] Acquire room_lock (se nd_to_all)
[12:25:04] [INFO] [LOCK] Release room_lock (se nd_to_all)
[12:25:05] [INFO] [COUNTDOWN] Thời gian còn lại: 14 giây...
[12:25:06] [INFO] [COUNTDOWN] Thời gian còn lại: 13 giây...
[12:25:07] [INFO] [COUNTDOWN] Thời gian còn lại: 12 giây...
[12:25:08] [INFO] [LOCK] Acquire price_lock (B ID)
[12:25:08] [INFO] [TIMER] Timer đã được RESET về 15s
[12:25:08] [INFO] [COUNTDOWN] Thời gian còn lại: 15 giây...
[12:25:08] [INFO] [LOCK] Release price_lock (B ID)
[12:25:08] [INFO] [SERVER] [✓] Chấp nhận BID 3000000 từ 'phuong'
[12:25:08] [INFO] [LOCK] Acquire room_lock (se nd_to_all)
[12:25:08] [INFO] [LOCK] Release room_lock (se nd_to_all)
[12:25:09] [INFO] [COUNTDOWN] Thời gian còn lại: 14 giây...

(base) haphuong@soclonlon test % python client.py
CLIENT ĐẦU GIÁ TRỰC TUYẾN
=====
Nhập tên của bạn: phuong
[✓] Đã kết nối tới server localhost:5000
Nhập giá bid (hoặc 'quit' để thoát):
> Chào mừng! Món đồ: 'MacBook Pro M4' | Giá khởi điểm: 1000000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 1000000 | Người dẫn đầu: chưa có ai
Nhập giá bid (hoặc 'quit' để thoát):
[✓] mai đã tham gia phòng
Nhập giá bid (hoặc 'quit' để thoát): 10000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] Lỗi: Giá quá thấp (hiện tại: 1000000)
Nhập giá bid (hoặc 'quit' để thoát): 1000000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] Lỗi: Giá quá thấp (hiện tại: 1000000)
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 1900000 | Người dẫn đầu: mai
Nhập giá bid (hoặc 'quit' để thoát): 3000000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 3000000 | Người dẫn đầu: phuong
Nhập giá bid (hoặc 'quit' để thoát):

(base) haphuong@soclonlon test % python client.py
CLIENT ĐẦU GIÁ TRỰC TUYẾN
=====
Nhập tên của bạn: mai
[✓] Đã kết nối tới server localhost:5000
Nhập giá bid (hoặc 'quit' để thoát):
> Chào mừng! Món đồ: 'MacBook Pro M4' | Giá khởi điểm: 1000000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 1000000 | Người dẫn đầu: chưa có ai
Nhập giá bid (hoặc 'quit' để thoát): 100000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] Lỗi: Giá quá thấp (hiện tại: 1000000)
Nhập giá bid (hoặc 'quit' để thoát): 1900000
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 1900000 | Người dẫn đầu: mai
Nhập giá bid (hoặc 'quit' để thoát):
[✓] GIÁ MỚI: 3000000 | Người dẫn đầu: phuong
Nhập giá bid (hoặc 'quit' để thoát):
```

Figure 4: The countdown timer explicitly resetting from 4s back to 15s.

5.4 Test Case 3: Concurrency & Stress Testing

- **Operation:** To evaluate the effectiveness of the Mutex mechanism when subjected to a high volume of concurrent requests, aiming to expose potential race conditions.
- **Execution:** The `stress_test.py` script deployed 30 automated bots using a synchronization barrier. Each bot sent 100 bids simultaneously, resulting in 3,000 requests hitting the server within 3 seconds.
- **Result:** All 3,000 bids were processed in 3.27 seconds, with no server crash or unexpected disconnection. The highest bid sent by any bot (1,101,000 VND) matched the final price accepted by the server exactly (1,101,000 VND, leading bidder: `stress_bot_3`), and the script reported **PASS**. This indicates that `price_lock` correctly serialized all concurrent bid updates: no lower bid overwrote a higher one.
- **Evidence:**

```

1  """
2  SERVER cho hệ thống Đấu giá Trực tuyến (Live Auction)
3  Môn: Công nghệ Mạng và Truyền thông
4  Bài tập: Exercise 08 - Live Online Auction Bidding System
5  """
6
7
8

```

```

[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_26 rời phòng. Còn lại: 5 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_9 rời phòng. Còn lại: 4 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_2 rời phòng. Còn lại: 3 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_17 rời phòng. Còn lại: 2 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_0 rời phòng. Còn lại: 1 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [LOCK] Acquire room_lock (disconnect cleanup)
[12:32:39] [INFO] [SERVER] stress_bot_15 rời phòng. Còn lại: 0 client
[12:32:39] [INFO] [LOCK] Release room_lock (disconnect cleanup)
[12:32:39] [INFO] [COUNTDOWN] Thời gian còn lại: 12 giây...
[12:32:40] [INFO] [COUNTDOWN] Thời gian còn lại: 11 giây...
[12:32:42] [INFO] [COUNTDOWN] Thời gian còn lại: 10 giây...
[12:32:43] [INFO] [COUNTDOWN] Thời gian còn lại: 9 giây...
[12:32:44] [INFO] [COUNTDOWN] Thời gian còn lại: 8 giây...
[12:32:45] [INFO] [COUNTDOWN] Thời gian còn lại: 7 giây...
[12:32:46] [INFO] [COUNTDOWN] Thời gian còn lại: 6 giây...
[12:32:47] [INFO] [COUNTDOWN] Thời gian còn lại: 5 giây...
[12:32:48] [INFO] [COUNTDOWN] Thời gian còn lại: 4 giây...
[12:32:49] [INFO] [COUNTDOWN] Thời gian còn lại: 3 giây...
[12:32:50] [INFO] [COUNTDOWN] Thời gian còn lại: 2 giây...
[12:32:51] [INFO] [COUNTDOWN] Thời gian còn lại: 1 giây...
[12:32:52] [INFO] [LOCK] Acquire price_lock (end_auction)
[12:32:52] [INFO] [LOCK] Release price_lock (end_auction)
[12:32:52] [INFO] [LOCK] Acquire room_lock (send_to_all)
[12:32:52] [INFO] [SERVER] PHÂN BỐ GIÁ KẾT THÚC
[12:32:52] [INFO] [SERVER] Người thắng: stress_bot_3 với giá 1101000
[12:32:52] [INFO] [SERVER] Cho 5 giây để client kịp nhận thông báo WIN...
[12:32:57] [INFO] [LOCK] Acquire room_lock (server shutdown)
[12:32:57] [INFO] [LOCK] Release room_lock (server shutdown)
[12:32:57] [INFO] [SERVER] Đã tắt server. Tạm biệt!
[12:32:57] [INFO] [SERVER] Port 5000 đã được giải phóng.
(base) haphuong@soclonton test %

```

```

python stress_test.py
[CLIENT] Đã đóng kết nối. Tạm biệt!
(base) haphuong@soclonton test % python stress_test.py

STRESS TEST - 30 clients x 100 bids
=====
Server: localhost:5000
Tổng số bid dự kiến: 3000

KẾT QUẢ STRESS TEST
=====
Thời gian test: 3.27 giây
Tổng số bid đã gửi: 3000
Giá cao nhất trong các bid đã gửi: 1101000
Giá cao nhất server công nhận: 1101000
Người dẫn đầu cuối cùng: stress_bot_3

PASS: Server công nhận đúng giá cao nhất!
-> Mutex hoạt động đúng, không có race condition.
(base) haphuong@soclonton test %

```

Figure 5: Stress test results showing successful synchronization of 3,000 concurrent bids.

5.5 Test Case 4: Fault Tolerance (Client Disconnection)

- **Operation:** To verify that the server continues the auction when a client leaves mid-auction, under two conditions of increasing severity.
- **Execution:**
 - (a) Controlled exit: with three bidders in the room and about 13 seconds remaining, one client (mai) terminated the client program using **Ctrl+C**, allowing the client to close its own socket.
 - (b) Process termination: in a second run with three bidders (hp, mai, na), after hp's bid of 4,500,000 VND had reset the countdown, the terminal running mai's client was closed with about 12 seconds remaining. This killed the process without any client-side cleanup, and the operating system closed the socket on its behalf.
- **Result:** In both cases the server detected the closed connection, acquired **room_lock**, and removed the socket without crashing. In (a), the countdown continued (12s, 11s, 10s...) and the remaining clients kept bidding until na won at 5,000,000 VND. In (b), the server logged mai's departure at 14:40:09 ("2 bidders remaining") with no error entry, indicating that the operating system closed the connection gracefully with a FIN on the process's behalf; the countdown continued without

interruption (11s, 10s...) and the remaining clients hp and na were unaffected, with the auction concluding normally (hp won at 4,500,000 VND).

- **Evidence:** Figures 6 and 7 (case a); Figure 8 (case b).

```

def main():
    sock.sendall(' auction | amount | 11 - encode | 11 - 0 ')
    except (BrokenPipeError, OSError):
        print("\n[CLIENT] không gửi được, server có thể đã tắt.")
        break
    except KeyboardInterrupt:
        print("\n[CLIENT] Client tạm thời disconnected, thoát...")
    finally:
        running = False
        try:
            sock.close()

```

```

(base) haphuong@soclontest test % py
thon server.py
[14:40:05] [INFO] [LOCK] Acquire price_lock (B
ID)
[14:40:05] [INFO] [TIMER] Timer đã được RESET
về 15s
[14:40:05] [INFO] [COUNTDOWN] Thời gian còn lại:
15 giây...
[14:40:05] [INFO] [LOCK] Release price_lock (B
ID)
[14:40:05] [INFO] [SERVER] [✓] Chấp nhận BID 45000
00
[14:40:05] [INFO] [LOCK] Acquire room_lock (se
nd_to_all)
[14:40:05] [INFO] [LOCK] Release room_lock (se
nd_to_all)
[14:40:05] [INFO] [COUNTDOWN] Thời gian còn lại:
14 giây...
[14:40:07] [INFO] [COUNTDOWN] Thời gian còn lại:
13 giây...
[14:40:08] [INFO] [COUNTDOWN] Thời gian còn lại:
12 giây...
[14:40:09] [INFO] [LOCK] Acquire room_lock (di
sconnect cleanup)
[14:40:09] [INFO] [SERVER] [✓] 'mai' đã rời phòng.
Còn lại: 2 người đang đấu giá.
[14:40:09] [INFO] [LOCK] Release room_lock (di
sconnect cleanup)
[14:40:09] [INFO] [COUNTDOWN] Thời gian còn lại:
11 giây...
[14:40:10] [INFO] [COUNTDOWN] Thời gian còn lại:
10 giây...
[14:40:11] [INFO] [COUNTDOWN] Thời gian còn lại:
9 giây...
[14:40:12] [INFO] [COUNTDOWN] Thời gian còn lại:
8 giây...

```

Figure 6: Server executing graceful degradation after a client disconnects unexpectedly (case a).

```

(base) haphuong@soclontest test % py
thon server.py
price_lock (end_auction)
[20:41:25] [INFO] [LOCK] Release
price_lock (end_auction)
[20:41:25] [INFO] [LOCK] Acquire
room_lock (send_to_all)
[20:41:25] [INFO] [LOCK] Release
room_lock (send_to_all)
[20:41:25] [INFO] [SERVER] [✓] PHIÊN
ĐẦU GIÁ KẾT THÚC
[20:41:25] [INFO] [SERVER] [✓] Người
thắng: na với giá 5000000
[20:41:25] [INFO] [SERVER] Chờ 5 gi
ây để client kịp nhận thông báo WIN
...
[20:41:30] [INFO] [LOCK] Acquire
room_lock (server shutdown)
[20:41:30] [INFO] [LOCK] Acquire
room_lock (disconnect cleanup)
[20:41:30] [INFO] [LOCK] Acquire
room_lock (disconnect cleanup)
[20:41:30] [INFO] [LOCK] Release
room_lock (server shutdown)
[20:41:30] [INFO] [LOCK] Release
room_lock (disconnect cleanup)
[20:41:30] [INFO] [LOCK] Release
room_lock (disconnect cleanup)
[20:41:30] [INFO] [SERVER] Đã tắt s
erver. Tạm biệt!
[20:41:30] [INFO] [SERVER] Port 500
0 đã được giải phóng.
(base) haphuong@soclontest test %

```

Figure 7: Remaining clients successfully continuing the bidding process and resolving the final winner (case a).

```

[12:42:48] [INFO] [COUNTDOWN] Thời gian còn lại: 3 giây...
[12:42:49] [INFO] [LOCK] Acquire room_lock (di sconnect cleanup)
[12:42:49] [INFO] [SERVER] 'mai' rời phòng. Còn lại: 1 client.
[12:42:49] [INFO] [LOCK] Release room_lock (di sconnect cleanup)
[12:42:49] [INFO] [COUNTDOWN] Thời gian còn lại: 2 giây...
[12:42:50] [INFO] [COUNTDOWN] Thời gian còn lại: 1 giây...
[12:42:51] [INFO] [LOCK] Acquire price_lock (e nd_auction)
[12:42:51] [INFO] [LOCK] Release price_lock (e nd_auction)
[12:42:51] [INFO] [LOCK] Acquire room_lock (se nd_to_all)
[12:42:51] [INFO] [LOCK] Release room_lock (se nd_to_all)
[12:42:51] [INFO] [SERVER] 🏆 PHIÊN ĐẦU GIÁ KẾT T HỨC
[12:42:51] [INFO] [SERVER] 🏆 Người thắng: na với giá 30000000
[12:42:51] [INFO] [SERVER] Chờ 5 giây để client k ịp nhận thông báo WIN...
[12:42:56] [INFO] [LOCK] Acquire room_lock (se rver shutdown)
[12:42:56] [ERROR] [SERVER] Client 'na' mất kết n ối
[12:42:56] [INFO] [LOCK] Release room_lock (se rver shutdown)
[12:42:56] [INFO] [LOCK] Acquire room_lock (di sconnect cleanup)
[12:42:56] [INFO] [LOCK] Release room_lock (di sconnect cleanup)
[12:42:56] [INFO] [SERVER] Đã tắt server. Tạm biệ t! 🏆
[12:42:56] [INFO] [SERVER] Port 5000 đã được giải phóng.
(base) haphuong@soclonton test %

```

Figure 8: Server logs showing process termination handled gracefully without error, and countdown continuing (case b).

5.6 Test Case 5: Message Framing over TCP

- **Operation:** To verify that the newline-based framing (Section 3.2) correctly reconstructs application messages regardless of how TCP delivers the byte stream, including multi-byte UTF-8 characters split across segments.
- **Execution:** A custom script (`framing_test.py`) opened raw TCP connections to the server with Nagle’s algorithm disabled (`TCP_NODELAY`), so that each `send()` call was transmitted immediately, and deliberately produced three delivery patterns:
 1. *Sticky packets:* a JOIN and a BID command concatenated into a single 30-byte `send()` call (`JOIN|framing_test\nBID|1500000\n`).
 2. *Fragmented packets:* a single `BID|1600000` command split across three `send()` calls of 2, 5 and 5 bytes, sent 0.5 seconds apart.
 3. *Split UTF-8 character:* a second client joined with the Vietnamese name “Phương”, split into two `send()` calls (8 and 6 bytes) exactly between the two bytes of the character “ư” (`0xC6 0xB0`).
- **Result:** All three cases were handled correctly, with no parsing or decoding errors in the server log.
 1. The server separated the two concatenated commands and processed them in order: the client received WELCOME, then an UPDATE with the current price, then `UPDATE|1500000|framing_test`, confirming that the bid was accepted.

- The fragments were buffered until the terminating newline arrived; the bid was then processed exactly once, as confirmed by the `UPDATE|1600000|framing_test` broadcast immediately after the third fragment. No partial value (such as a bid of 16) was ever interpreted.
- The incremental UTF-8 decoder held the incomplete first byte until the second fragment arrived. The server logged the new bidder “Phường” with the name intact, and the other client received the corresponding INFO notification, correctly encoded.

The auction subsequently concluded normally, with `framing_test` winning at 1,600,000 VND, which further confirms that the fragmented bid had been recorded as a single, complete value.

- **Evidence:** Figure 9.

```

1  =====
2  TEST MESSAGE FRAMING cho hệ thống Đấu giá Trục tuyến
3  Kiểm tra server tách đúng bản tin khi TCP:
4  (1) dồn nhiều bản tin vào 1 lần gửi (sticky packets)
5  (2) chia 1 bản tin thành nhiều lần gửi (fragmented packets)
6  (3) cắt đôi 1 ký tự UTF-8 nhiều byte (tên tiếng Việt)
7  Cách chạy: khởi động server.py M01, rồi chạy: python framing_test.py
8  =====
9
10
11
12 import socket
13 import threading
14 import time
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Figure 9: Message framing test: sticky packets, fragmented packets and a split UTF-8 character (right), and the corresponding server log (left).

Chap 6

Conclusion

6.1 Achievements

The project delivered a working live auction system built directly on TCP sockets, meeting every requirement of the assignment:

- **Auction management:** the server announces the item and starting price, accepts only strictly higher bids, and broadcasts every new highest bid and leading bidder to the whole room (Test Case 1, Section 5.2).
- **Countdown timer:** a 15-second countdown restarts on every accepted bid and declares the winner when it expires (Test Case 2, Section 5.3).
- **Shared-state synchronization:** under 3,000 concurrent bids from 30 clients, the final price matched the highest bid sent, showing that `price_lock` correctly serialized all updates (Test Case 3, Section 5.4).
- **Fault tolerance:** the auction continued unaffected when a client exited or its process was killed mid-auction (Test Case 4, Section 5.5).
- **Protocol robustness:** the application-layer framing correctly handled sticky packets, fragmented packets and multi-byte UTF-8 characters split across segments (Test Case 5, Section 5.6).
- **Real-time client:** a two-thread client displays price updates while the user is typing (Section 4.5).

6.2 Technical Challenges & Solutions

- **Timer state conflicts:** each bid spawns a new countdown thread, so stale threads could close the auction prematurely. A `timer_version` token lets superseded threads exit, and the version is re-validated inside `price_lock` before closing, eliminating a residual check-then-act race (Section 4.4).
- **Message boundaries and split characters:** decoding each `recv()` result independently would misinterpret merged or fragmented messages, and would raise a decoding error whenever a multi-byte UTF-8 character was split between two segments. Both server and client therefore buffer incoming data, decode it incrementally, and process only complete newline-terminated messages (Sections 3.2

and 4.5, verified in Test Case 5).

- **Dictionary mutation during iteration:** removing a disconnected client while broadcasting raised a `RuntimeError`. This was resolved by collecting broken sockets in a temporary list and removing them after the iteration, inside the same `room_lock` block (Section 4.2).
- **Blocking console input:** a single-threaded client could not show new prices while waiting for the user to type. Splitting input and reception into two threads solved this (Section 4.5).
- **Socket closure exceptions (Errno 9):** “Bad file descriptor” errors appeared at shutdown because the main thread closed client sockets while their handler threads were still blocked in `recv()`. The current implementation mitigates this with a short delay before shutdown, which gives clients time to receive the final `WIN` message; a complete fix is listed as future work.

6.3 Known Limitations

The following limitations are known and were deliberately left outside the scope of this assignment:

- **Silent disconnections are not detected:** a client that loses network connectivity without closing its socket (for example, a cable pull or Wi-Fi loss) remains in the room indefinitely, because no heartbeat or TCP keepalive is implemented.
- **Broadcast ordering:** updates are broadcast after `price_lock` is released. If two bids are accepted almost simultaneously, their `UPDATE` messages may reach clients in reverse order, briefly showing an outdated price until the next update.
- **Slow receivers:** `send_to_all()` sends to clients one after another while holding `room_lock`, so a client that reads very slowly can delay broadcasts to everyone else (Section 3.4).
- **No message length limit:** a client that sends a long stream without a newline makes the server’s buffer grow without bound.
- **Single session per run:** each server run hosts one auction for one item and shuts down afterwards.
- **Timer design:** each accepted bid spawns a new countdown thread, and the one-second tick accumulates slight timing drift.
- **Console interface:** the client cannot interrupt a pending `input()` call, so the user must press Enter once more after the auction ends.

- **Test coverage:** all tests ran over the loopback interface on a single machine; behaviour under real network latency and packet loss was not evaluated.
- **Security:** there is no authentication and no encryption; any client can join under any unused name.

6.4 Future Work

- **Liveness detection:** add a periodic heartbeat (or enable TCP keepalive) so that silently disconnected clients are removed.
- **Ordered, non-blocking broadcasts:** give each client its own outgoing message queue served by a dedicated sender, and attach a sequence number to every `UPDATE` so clients can discard stale ones.
- **Graceful shutdown:** replace the fixed delay with `shutdown(SHUT_RDWR)`, letting each handler thread observe end-of-stream and close its own socket.
- **Scalability:** replace thread-per-connection with I/O multiplexing (`selectors` or `asyncio`), and replace per-bid countdown threads with a single timer comparing the current time against a shared deadline.
- **Robustness:** enforce a maximum message length and disconnect clients that exceed it.
- **Features:** multiple parallel auction rooms, persistent bidding history in a database (e.g., SQLite or PostgreSQL), TLS encryption for use over public networks, and a graphical client (e.g., Tkinter or PyQt) whose event loop would also remove the console input limitation.

6.5 Concluding Remarks

Beyond meeting the functional requirements, the project showed that most of the difficulty in network programming lies not in sending and receiving data, but in the timing between concurrent activities: bids arriving together, a timer expiring as a bid is accepted, or a message arriving in pieces. The most instructive bug, the residual timer race of Section 4.4, passed every ordinary test and was only found by reasoning carefully about what can happen between a check and the action that depends on it. The limitations listed above are documented deliberately, and each points to a concrete next step.